

SESIÓN 02

Python + Flask

Primera API

Construyendo el backend de TaskFlow

Objetivos de la Sesión

- Comprender qué es una API REST
- Conocer los métodos HTTP: GET, POST, PUT, DELETE
- Crear un servidor web básico con Flask
- Implementar rutas que respondan a peticiones
- Probar APIs usando Thunder Client

¿Qué es una API?



Application Programming Interface

Una API es un conjunto de reglas que permite que diferentes programas se comuniquen entre sí.

 Analogía del Restaurante:

- Tú (cliente) → haces un pedido
- Mesero (API) → lleva tu pedido a la cocina
- Cocina (servidor) → prepara la comida
- Mesero → te trae la respuesta

Métodos HTTP

Método	Acción	Ejemplo
GET	Leer/Obtener	GET /api/users - Lista usuarios
POST	Crear	POST /api/users - Crea usuario
PUT	Actualizar	PUT /api/users/1 - Actualiza usuario 1
DELETE	Eliminar	DELETE /api/users/1 - Elimina usuario 1

Códigos de Estado HTTP

Código	Significado	Uso
200	OK	Operación exitosa
201	Created	Recurso creado
400	Bad Request	Datos incorrectos
401	Unauthorized	No autenticado
404	Not Found	No existe
500	Server Error	Error del servidor

¿Qué es Flask?



Microframework de Python para web

Flask es un framework minimalista que te da libertad para estructurar tu aplicación.

- ✓ Fácil de aprender
- ✓ Documentación excelente
- ✓ Comunidad grande
- ✓ Usado por Netflix, Reddit, Lyft

Primera Aplicación Flask

```
# backend/app.py
from flask import Flask, jsonify
from flask_cors import CORS

app = Flask(__name__)
CORS(app) # Permite conexiones del frontend

@app.route("/api/health", methods=["GET"])
def health_check():
    return jsonify({
        "status": "ok",
        "message": "TaskFlow API v1.0"
    })

if __name__ == "__main__":
    app.run(debug=True, port=5000)
```

CRUD de Usuarios - GET

```
# Lista en memoria (temporal)
users = [
    {"id": 1, "nombre": "Admin", "email": "admin@taskflow.com"},
    {"id": 2, "nombre": "Juan", "email": "juan@email.com"}
]

@app.route("/api/users", methods=["GET"])
def listar_usuarios():
    return jsonify(users)

@app.route("/api/users/<int:user_id>", methods=["GET"])
def obtener_usuario(user_id):
    for user in users:
        if user["id"] == user_id:
            return jsonify(user)
    return jsonify({"error": "No encontrado"}), 404
```

CRUD de Usuarios - POST

```
@app.route("/api/users", methods=["POST"])
def crear_usuario():
    global next_id
    data = request.get_json()

    if not data or not data.get("nombre"):
        return jsonify({"error": "Nombre requerido"}), 400

    nuevo = {
        "id": next_id,
        "nombre": data["nombre"],
        "email": data.get("email", ""),
        "rol": data.get("rol", "usuario")
    }
    users.append(nuevo)
    next_id += 1
    return jsonify(nuevo), 201
```

API de Usuarios Completa

Objetivo: Crear una API REST completa con validaciones

1. Crear entorno virtual: `python -m venv venv`
2. Activar: `venv\Scripts\activate` (Windows)
3. Instalar: `pip install flask flask-cors`
4. Implementar GET `/api/users` (listar)
5. Implementar POST `/api/users` (crear con validación)
6. Implementar PUT `/api/users/<id>` (actualizar)
7. Implementar DELETE `/api/users/<id>` (eliminar)
8. Probar con Thunder Client

Probar con Thunder Client

- 1. Abrir Thunder Client (ícono del rayo)
- 2. Nueva Request > GET > `http://localhost:5000/api/users`
- 3. Send y verificar respuesta
- 4. POST > `http://localhost:5000/api/users`
- 5. Body > JSON: `{"nombre": "Test", "email": "test@test.com"}`
- 6. Send y verificar código 201



Resumen de la Sesión

- ✓ Entendemos qué es una API REST
- ✓ Conocemos los métodos HTTP y códigos de estado
- ✓ Creamos servidor Flask functional
- ✓ Implementamos CRUD básico de usuarios
- ✓ Probamos endpoints con Thunder Client

Próxima sesión: CRUD Completo + Estructura MVC